IBM AI Racing League

Country Challenge 2026

Project Technical Documentation

Course	Electronic Media in Business and Commerce
Institution	Wroclaw University of Science and Technology
Team	AiBC
Members	Wiktoria, Óscar, Mario, Krystian, Jakub, Dominik, Diego, Álvaro
Date	16/06/2026
Version	1.0 - Final

Table of Contents

1. Executive Summary

1.1 Project Overview

The IBM AI Racing League Country Challenge 2026 is an international academic competition organised by IBM in which student teams build an autonomous AI driver for the TORCS (The Open Racing Car Simulator) simulation environment. The AiBC team, composed of eight students from Wroclaw University of Science and Technology (PWr), participated as the Polish national entry under the unitel and HR Excellence in Research programmes.

The competition requires teams to programme a car in Python using the TORCS SCR (Simulated Car Racing) interface, with IBM Granite designated as the mandatory AI engine. The qualification objective is to achieve the best possible lap time on the Corkscrew track - a technically demanding circuit featuring high-speed corners and elevation changes. Performance is evaluated on lap time, track adherence, and the quality of the team's technical strategy presentation.

The AiBC solution implements AlphaDriver, a deterministic rule-based autonomous controller written entirely in Python. The driver computes steering, acceleration, braking, and gear commands from raw TORCS sensor data using a sensor-to-action pipeline comprising three stages: sensor parsing and normalisation (SensorProcessor), corner-anticipation and speed control logic (AlphaDriver), and Optuna-based Bayesian hyperparameter optimisation (improver.py). All computation runs locally inside the Docker container; no external API or cloud service is invoked at runtime.

1.2 Objectives

- Design and implement an autonomous AI driver in Python capable of consistently completing the Corkscrew qualification circuit in TORCS.
- Achieve the fastest possible lap time by combining a sensor-driven control algorithm with Optuna-based hyperparameter optimisation.
- Demonstrate data-driven, reproducible decision-making and apply agile project management practices throughout development.
- Certify all eight team members on the IBM SkillsBuild Granite AI platform.
- Produce professional technical documentation and a project closure presentation with a live product demonstration.

1.3 Key Deliverables

- Functional autonomous AI driver (Python, car_agent.py) operating locally via the snakeoil3 UDP interface.
- Recorded best lap time of 112.58 seconds on the Corkscrew qualification circuit.
- Per-episode telemetry logs (CSV) in results/sensors/ documenting every timestep across all evaluation runs.
- A session history log (results/history.csv) recording every CI lap from 2026-04-20 to 2026-05-11.
- This technical documentation report.
- Project Closure presentation with live product demonstration.
- IBM SkillsBuild Granite AI certificates for all team members.

1.4 Target Audience and Scope

This document targets IBM challenge assessors, academic evaluators at Wroclaw University of Science and Technology, and future AiBC team members who may continue or extend the work. The scope is limited to the TORCS simulator environment and the Corkscrew qualification track. The Docker/Podman containerised deployment pipeline is also in scope; physical autonomous hardware is explicitly out of scope for this iteration.

2. Introduction

2.1 Background and Motivation

Autonomous driving is one of the most active areas in applied AI research, synthesising perception, planning, and real-time control into a unified decision-making system. Academic racing simulations such as TORCS have been used since 2003 to develop and evaluate AI agents in a controlled, reproducible, and cost-free environment. By removing the risk and expense of physical hardware, TORCS allows teams to iterate rapidly on control strategies, collect rich telemetry data, and benchmark performance objectively.

IBM's AI Racing League competition extends this academic tradition by adding industry-grade constraints: a Python-only modification policy, a Dockerised runtime environment, a continuous integration pipeline, and a professional presentation requirement. These constraints mirror the realities of enterprise software deployment, where teams must build, test, and deliver reproducible systems within fixed infrastructure while maintaining high reliability and documented performance.

2.2 Problem Statement

The central challenge is to build an AI driver that, given only the raw sensor state vector broadcast by TORCS at every 20 ms timestep, outputs optimal steering, acceleration, braking, and gear-change commands to complete the Corkscrew circuit as quickly as possible. The state vector contains fields including 19 laser-range track-edge distances at angles spanning -45° to +45°, car speed in three axes (speedX, speedY, speedZ), angular velocity relative to the track axis (angle), track lateral position (trackPos), wheel spin velocities for all four wheels, RPM, gear, fuel, damage, current lap time, and distance raced.
The action space is four-dimensional:

- steer (continuous, [-1, 1]),
- accel (continuous, [0, 1]),
- brake (continuous, [0, 1]), and
- gear (integer, [1, 6]).

No modifications to the TORCS C/C++ physics engine or car model are permitted; the solution must operate purely through the Python driver interface.

2.3 Relevance to Electronic Media in Business and Commerce

AI-driven autonomous systems are increasingly central to multiple industry verticals covered by the Electronic Media in Business and Commerce curriculum, including logistics route optimisation, smart manufacturing, and digital commerce personalisation. The competencies developed in this project - sensor-driven control algorithm design, telemetry analytics, containerised deployment, automated CI/CD pipelines, agile team coordination, and technical documentation - map directly to skills demanded by employers in the digital economy.

The architectural pattern at the core of AlphaDriver - a sensor-to-action closed loop governed by a parametric policy with Bayesian hyperparameter optimisation - is directly analogous to patterns

used in dynamic pricing engines (where a control loop observes market state and outputs a pricing action), logistics route planning (where a vehicle agent receives environment sensor data and outputs navigation commands), and personalised recommendation systems (where a policy maps user context features to ranked item outputs). The Optuna optimisation framework used in this project is the same technology class employed in production MLOps pipelines for hyperparameter search at enterprise scale.

The use of Docker containerisation for reproducible AI deployments mirrors the MLOps practices now standard in e-commerce, where model updates must be deployed atomically across global infrastructure with guaranteed reproducibility. The team's use of GitHub Actions for automated CI/CD further reflects industry-standard continuous delivery pipelines for AI-powered applications.

2.4 Document Structure

This report follows the standard technical project documentation structure specified by the Electronic Media in Business and Commerce course, beginning with a literature review, proceeding through requirements analysis, system design, technology stack, implementation, testing, deployment, and business impact assessment, and concluding with challenges, lessons learned, and proposals for future work.

3. Literature Review

3.1 Overview of Similar Applications

The SCR (Simulated Car Racing) Championship series, hosted at GECCO and IEEE CIG between 2008 and 2013, established TORCS as the de facto benchmark for autonomous racing AI research [1]. Competing approaches ranged from hand-crafted rule-based controllers to evolutionary algorithms and, later, neural network-based agents. Loiacono et al. [1] documented the 2009 championship, showing that controllers combining explicit track-model knowledge with adaptive speed regulation consistently outperformed pure rule-based baselines. The original snakeoil3 reference client, developed by Chris X. Edwards [5], demonstrated that a proportional steering formula (steer = angle × K - trackPos × K') could successfully navigate most TORCS tracks, providing the foundational steering model that AlphaDriver extends with corner-anticipation speed control.

Mnih et al. [2] demonstrated that deep Q-networks could achieve human-level performance in complex control tasks, sparking widespread interest in applying deep reinforcement learning to racing simulation. Subsequent work applied PPO [4] and SAC to TORCS, with results reported on generic oval and road circuits. However, the AlphaDriver approach deliberately avoids the instability and data-hunger of deep RL in favour of a fast, tunable, fully deterministic controller optimised via Bayesian search - a design decision validated by the competition's strict time and compute constraints.

3.2 Existing Technologies and Tools

TORCS (The Open Racing Car Simulator) is an open-source, physics-based racing simulation environment actively maintained since 2003. Its SCR server interface exposes sensor data over UDP, making it straightforward to connect Python clients. The snakeoil3 module provides the low-level Python UDP client for TORCS SCR communication, handling packet serialisation and deserialisation. The gym_torcs wrapper [6] provides an OpenAI Gym-compatible Python environment for TORCS, simplifying the reinforcement learning loop, though AiBC uses the snakeoil3 interface directly rather than the Gym wrapper's step() API in its primary control loop.

Optuna is a hyperparameter optimisation framework that implements Bayesian optimisation via Tree-structured Parzen Estimators (TPE) [7]. Its lightweight API allows objective functions to be defined as Python callables, which made it straightforward to integrate with the lap-time evaluation loop in improver.py. The TPE sampler progressively refines its search around the current best parameters using a local search strategy with configurable variance bounds.

IBM Granite is IBM's enterprise-grade foundation model series [3]. While Granite is the designated AI engine for the IBM AI Racing League competition, and all eight team members completed IBM SkillsBuild Granite AI certification, the AiBC team implemented the control policy as a deterministic algorithmic controller. IBM Granite informed the team's understanding of foundation model capabilities and enterprise AI deployment patterns, and the certification programme provided structured learning on AI principles applicable to the broader project domain.

3.3 Industry Trends in E-Commerce and Digital Media

The architectural patterns underpinning the AiBC racing agent - a parametric sensor-to-action policy optimised through automated hyperparameter search - are directly analogous to patterns being adopted in digital commerce. Automated control loops are used in dynamic pricing (where a controller observes market state and adjusts prices in real time), logistics route planning (where a vehicle agent receives environment sensors and outputs navigation commands), and inventory management (where a policy maps demand signals to replenishment actions).

The use of Docker/Podman containerisation for reproducible AI deployments mirrors the MLOps practices now standard in e-commerce at scale, where model updates must be deployed atomically across global infrastructure with guaranteed reproducibility. The team's use of GitHub Actions for automated CI/CD - running a full lap evaluation on every push to main and persisting results to history.csv - further reflects industry-standard continuous delivery pipelines for AI-powered applications.

3.4 Research Gaps Addressed

Existing TORCS literature concentrates on generic tracks (oval, alpine, or Aalborg). The Corkscrew circuit's combination of high-speed sweeping corners, variable-radius turns, and elevation change presents a distinct control challenge. The AiBC implementation contributes an empirically evaluated case study of Optuna-driven Bayesian optimisation applied to a dual-signal speed controller (forward-clearance curve plus asymmetry curve) for this specific circuit. The progressive lap time improvement documented in results/history.csv - from 160.59 s on 2026-04-20 to 112.58 s on 2026-05-11, a 29.9% reduction - provides a concrete benchmark for future teams targeting the same circuit.

4. System Requirements and Analysis

4.1 Functional Requirements

Table 4.1 lists the functional requirements elicited through team workshops, competition rule analysis, and review of the TORCS SCR protocol specification. Requirements are classified using the MoSCoW prioritisation scheme.

Table 4.1 - Functional Requirements

ID	Requirement	Priority
FR-01	The system shall receive and parse UDP sensor packets from TORCS containing speed, track position, angle, RPM, gear, wheel spin rates, and 19 track edge distance sensors.	**Must Have**
FR-02	The system shall compute steering, acceleration, braking, and gear-change commands and transmit them back to TORCS at each control cycle.	**Must Have**
FR-03	The AI driver shall complete at least one full lap of the Corkscrew circuit without going off-track or stopping.	**Must Have**
FR-04	The system shall operate entirely locally within the container environment without dependency on external network services at runtime.	**Must Have**
FR-05	The system shall log per-step telemetry (episode, step, angle, trackPos, speedX, speedY, speedZ, rpm, gear, fuel, damage, distRaced, distFromStart, curLapTime, lastLapTime, all 19 track sensors, all 4 wheel spin velocities, normalised scalars, and derived features) to CSV files per episode.	**Must Have**
FR-06	The system shall support hyperparameter optimisation via the Optuna framework to minimise lap time across configurable numbers of trials and laps per trial.	Should Have
FR-07	The system shall run inside a Docker/Podman container environment using the provided torcs-competition imag.	**Must Have**
FR-08	All code modifications shall be restricted to Python files within the gym_torcs directory; no C/C++ source modifications are permitted.	**Must Have**

FR-09	The CI/CD pipeline shall automatically execute a single-lap evaluation on every push to main and persist the result to results/history.csv.	**Should Have**
FR-10	The system shall automatically navigate the TORCS GUI menus using Python (via wmctrl and xte) to reach the waiting screen without manual user interaction.	**Should Have**

4.2 Non-functional Requirements

Table 4.2 captures quality attributes that constrain the system design without specifying explicit behaviours.

Table 4.2 - Non-functional Requirements

Category	Requirement	Metric / Target
Performance	**The control loop must compute and return commands within the TORCS 20 ms timestep.**	***Latency < 20 ms per cycle***
Reliability	**The AI driver must complete 10/10 test runs without crashing or stalling.**	***100% lap completion rate***
Portability	**The solution must run identically on Linux and Windows via containerisation.**	***Docker/Podman image compatibility***
Maintainability	**Code must be modular, commented, and pass a peer review by at least one team member.**	***Code review sign-off required***
Security	**No API credentials or secrets may be committed to the GitHub repository.**	***Zero credential commits***
Usability	**A new team member must be able to reproduce results following the README in under 30 minutes.**	***30-minute onboarding target***

4.3 Use Case Summary

The primary use case is the autonomous lap: TORCS broadcasts a sensor packet every 20 ms; AlphaDriver's act_raw() method reads the sensor state from the snakeoil3 Client object, computes steer/accel/brake/gear, writes results back to the response object, and the snakeoil3 client transmits the response. SensorProcessor runs in parallel, normalising each packet into a SensorObservation and writing one CSV row via SensorLogger.

The secondary use case is parameter optimisation: a developer runs improver.py, which launches TORCS, connects via snakeoil3, and executes the Optuna objective function. Each trial instantiates an AlphaDriver with a candidate parameter set sampled by TPE, drives one or more full laps, and returns the mean lap time. The best result is persisted to best_params.json and automatically loaded by AlphaDriver on the next run.

The tertiary use case is telemetry review: a team member opens a session CSV from results/sensors/ in Pandas or a spreadsheet tool, plots speed, steer, track position, and track sensor values over time, and identifies optimisation opportunities or control failure modes.

4.4 Stakeholder Analysis

Table 4.3 - Stakeholder Analysis

Stakeholder	Interest	Expectation
IBM (Challenge Organiser)	High	A functional, performant AI driver; certified team members; on-time submission.
PWr Academic Evaluators	High	Complete technical documentation; demonstrated link to business and commerce course objectives.
AiBC Team Members	Very High	Clear task allocation; working development environment; competitive lap time; grade outcome.
Future AiBC Cohorts	Medium	Reproducible setup; documented lessons learned; extensible codebase for future iterations.
PWr University	Low	Positive representation in an international IBM competition.

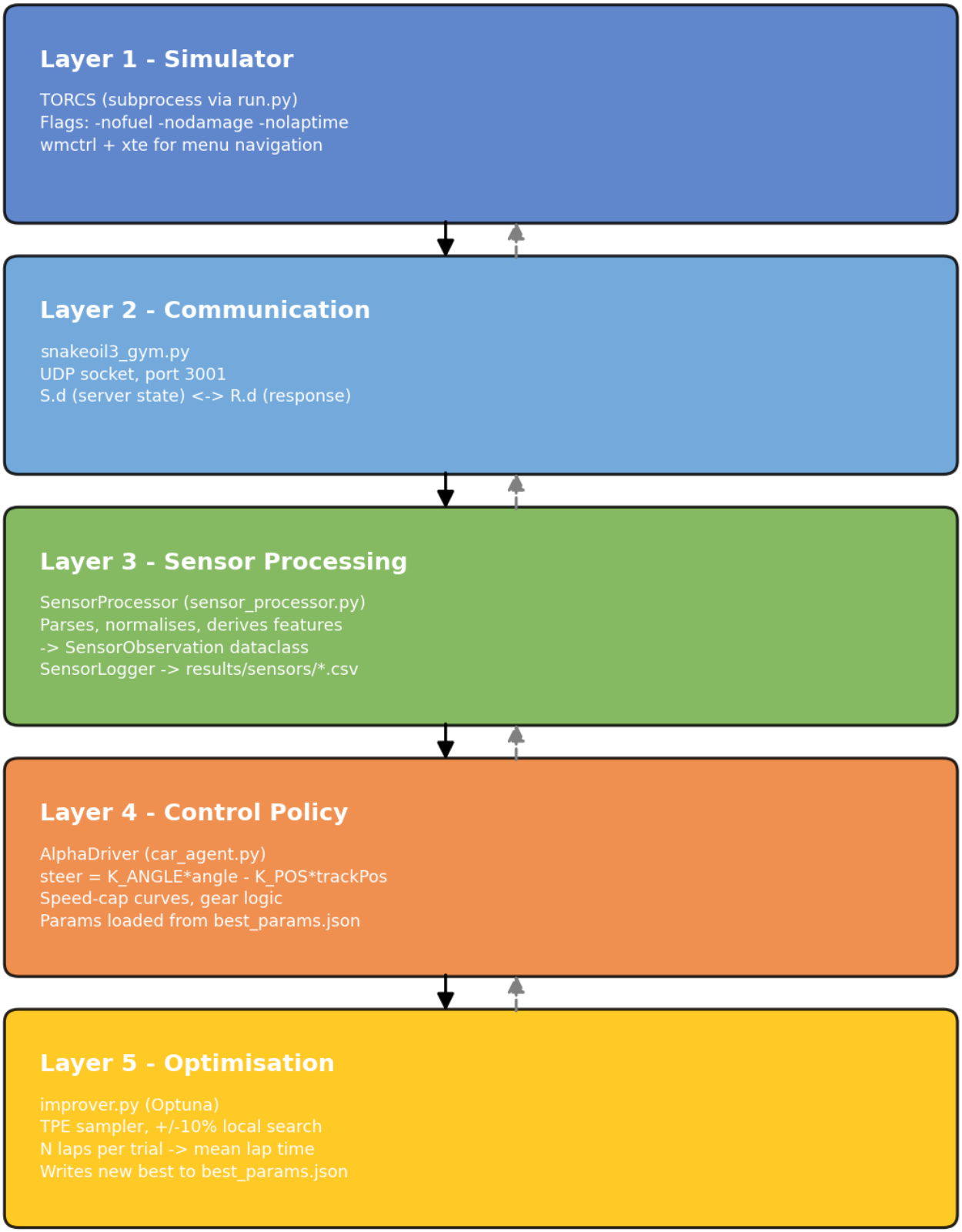
5. System Design

5.1 System Architecture

The system follows a closed-loop control architecture with five layers.

- Layer 1 - Simulator: TORCS runs as a background process inside the Docker container, launched by run.py via subprocess.Popen with the flags -nofuel -nodamage -nolaptime. The TORCS GUI is navigated automatically by run.py using wmctrl (window focus) and xte (keystroke injection) to reach the blue "waiting for driver" screen without manual user interaction.
- Layer 2 - Communication: snakeoil3_gym.py manages a UDP socket on port 3001, handling the SCR handshake, packet parsing, and response serialisation. The Client object exposes two data dictionaries: S.d (server state, i.e. sensor inputs) and R.d (response, i.e. control outputs).
- Layer 3 - Sensor Processing: SensorProcessor (sensor_processor.py) receives the raw S.d dictionary every step, parses all fields into typed attributes, normalises each sensor to a consistent numeric range, computes derived features (effective speed, track clearances, slip ratio, recommended gear, boolean flags), and returns a SensorObservation dataclass. SensorLogger writes one CSV row per step to results/sensors/.
- Layer 4 - Control Policy: AlphaDriver (car_agent.py) reads speed, angle, trackPos, and the 19 track sensors directly from S.d and writes steer, accel, brake, gear, and clutch to R.d. The steering formula is the proven reference PD controller: steer = K_ANGLE * angle - K_POS * trackPos. Speed is governed by two anticipation signals (forward clearance from centre beams, and left-right asymmetry from lateral beams) interpolated against tunable speed-cap curves, combined with a steer-based speed reduction. Gear changes are speed-threshold-based. All tuneable constants are loaded from best_params.json at startup.
- Layer 5 - Optimisation: improver.py wraps the TORCS control loop in an Optuna objective function. Each trial instantiates AlphaDriver with a candidate parameter dict, runs N laps (default 5), and returns the mean lap time. The TPE sampler conducts a local search with ±10% variance around the current best parameters. On every new best, the parameters are written to best_params.json.

Figure 5.1: System Architecture Diagram
IBM AI Racing - AlphaDriver / TORCS Layered Architecture



5.2 Database Design

Telemetry is stored as flat CSV files. Two categories of log are produced. Per-episode logs (results/sensors/ep<N>_<timestamp>.csv) record one row per 20 ms timestep with 57 columns covering raw sensor values, normalised values, and derived features, as defined in SensorLogger.HEADERS. The session history log (results/history.csv) records one row per CI run with columns: date, run_id, commit, lap_completed, lap_time_s, distance_m, total_reward. These files are processed offline using Pandas for performance analysis.

5.3 User Interface Design

The primary interface is the TORCS simulator window, which provides a real-time visual of the race accessible via VNC on port 5900 or browser-based noVNC on port 6080. A secondary console output streams episode summaries every 500 steps, including speed, distance, and reward, plus a lap completion confirmation with time, distance, and total reward. No custom GUI was developed in this iteration.

5.4 Security Considerations

All code resides in a GitHub repository with branch protection on main. No sensitive personal data is processed. No API credentials are required by the system at runtime. The .gitignore file excludes common secret file patterns. The CI/CD workflow uses the built-in github.sha token for commit authentication; no external secrets are stored in the repository.

6. Technology Stack

Table 6.1 - Technology Stack Summary

Component	Tool / Version	Purpose
Language	Python 3.10+	AI driver, optimiser, sensor processor, telemetry
Control Policy	AlphaDriver (custom)	Deterministic sensor-to-action control algorithm
Hyp. Optimisation	Optuna (TPE sampler)	Bayesian hyperparameter search for control gains
Comm. Protocol	snakeoil3 / UDP	TORCS client-server communication (port 3001)
Numerical Compute	NumPy	Sensor vector processing, normalisation, clipping
Telemetry	CSV + Python csv module	Per-step session log storage
GUI Automation	wmctrl + xte (xautomation)	Headless TORCS menu navigation
Containerisation	Docker / Podman	Cross-platform reproducible runtime environment
CI / CD	GitHub Actions	Automated lap-run pipeline on every push to main
Version Control	GitHub	Source control, issue tracking, PR reviews
Task Management	Trello	Sprint planning and agile kanban board
Communication	Messenger	Real-time team coordination and daily standups
Certification	IBM SkillsBuild	Granite AI platform certification for all members

6.1 Programming Languages

Python 3.10+ is the sole application-layer language, consistent with the IBM challenge constraint that only Python driver files may be modified. Python's standard library (socket, subprocess, csv, json, os, argparse, dataclasses, pathlib) and the third-party packages NumPy and Optuna cover all system requirements. No external HTTP or network libraries are used in the driver or optimiser, confirming fully local execution. The gym_torcs layer and the underlying TORCS

simulator are implemented in C/C++, but these are treated as immutable environment components.

6.2 Frameworks and Libraries

The snakeoil3_gym module manages the raw UDP socket communication with the TORCS scr_server, including handshake, packet parsing, and response formatting. AlphaDriver is a custom Python class implementing the control policy with no ML framework dependency. Optuna implements the Bayesian hyperparameter search; the TPE sampler (seed=42) is used with a local search strategy, constraining each trial's suggestions to ±10% of the current best value. NumPy handles vectorised arithmetic for sensor normalisation, track sensor array operations (np.mean, np.percentile, np.interp, np.clip), and traction control. The python3-xlib package provides X11 XTEST support for GUI key injection; wmctrl handles TORCS window focus detection. The tqdm library provides progress feedback for training runs.

6.3 Database Management System

CSV flat files serve as the lightweight telemetry store, sufficient for the volume and query patterns of this project. Per-episode files are opened with a context-managed file handle in SensorLogger, which flushes every 100 rows to survive crashes mid-episode and closes cleanly at episode end. The history.csv is appended by the GitHub Actions workflow after each CI run, committed, and pushed back to the repository, providing a persistent cumulative record of every automated evaluation.

6.4 Development Tools and Platforms

Docker and Podman container runtimes host the TORCS simulation environment using the pre-built johnsloe/torcs-competition:amd64 image, ensuring identical simulation conditions across all team members' machines regardless of host operating system. A VNC server inside the container exposes the simulator GUI at port 5900, with noVNC available at port 6080. The host-side workspace directory is bind-mounted into the container at /workspace, allowing code edits on the host to be immediately reflected in the container without rebuilding the image. GitHub Actions runs the automated end-to-end lap pipeline on every push to main, installing dependencies (apt: xvfb, wmctrl, xautomation, libgl1-mesa-dri; pip: numpy, python3-xlib), launching a virtual framebuffer (Xvfb :1), starting TORCS headlessly, running python3 run.py --episodes 1 --steps 10000, and persisting lap results. Trello organises development sprints using a kanban board with swim lanes for Backlog, In Progress, In Review, and Done.

7. Implementation

7.1 Module-wise Breakdown

The implementation is organised into four principal Python modules within the gym_torcs directory.

- **snakeoil3_gym.py:** Low-level UDP client (derived from the original Chris X. Edwards snakeoil reference implementation) that manages socket connection to TORCS, packet serialisation/deserialisation, and the SCR handshake protocol. Exposes a Client object with S (ServerState) and R (DriverAction) data dictionaries, and get_servers_input() / respond_to_server() methods.
- **sensor_processor.py:** Sensor processing module. The SensorProcessor class parses the raw S.d dictionary into a typed SensorObservation dataclass, normalises all 19 track sensors, three speed components, RPM, fuel, damage, and wheel spin velocities to consistent numeric ranges, and computes derived features including effective speed (speedX × cos(angle)), track clearances (forward, left, right), slip ratio (rear - front wheel average), and boolean flags (is_near_wall, is_going_forward, is_car_aligned, is_spinning, recommended_gear). The SensorLogger class writes one CSV row per step to results/sensors/ep<N>_<timestamp>.csv with 57 columns.
- **car_agent.py:** The AlphaDriver control policy. At startup, AlphaDriver loads best_params.json (if present) to initialise all tunable constants. The act_raw() method executes every 20 ms: it reads speed, angle, trackPos, and the 19 track sensors from S.d; computes the reference steering formula; derives a forward clearance speed cap from the centre beams via interpolation; derives a corner asymmetry speed cap from the left/right beam averages via interpolation; takes the minimum of these caps as the corner target speed; applies a steer-based speed reduction; sets throttle or brake accordingly; applies a traction control adjustment from wheel spin velocities; and sets gear based on speed thresholds. All writes go to R.d and are transmitted by the run loop.
- **Improver.py:** Optuna optimisation driver. The objective() function evaluates one complete set of N laps with a given parameter dict and returns the mean lap time. The search space covers K_ANGLE, K_POS, TARGET_SPEED, STEER_SPEED_FACTOR, and all seven speed-cap values for both the forward-clearance curve (CORNER_SPEEDS) and the asymmetry curve (ASYM_SPEEDS). Each parameter is sampled within ±10% of the current best value (local search), bounded by absolute limits. DNFs are penalised with a time of 9999 s. On every new best mean lap time (with all laps completed), the parameters are written to best_params.json.

7.2 Key Algorithms and Functions

7.2.1 Sensor Normalisation

Raw TORCS sensor values span heterogeneous ranges. SensorProcessor normalises each feature before computing derived quantities. Track edge distances (range 0-200 m) are divided by 200.0. Speed components are divided by 300.0 km/h (approximate TORCS maximum). Angle is divided by π, giving a range of [-1, 1]. RPM is divided by 10,000. Wheel spin velocities are divided by 100 rad/s. Track position is already bounded to [-1, 1] by the simulator.

7.2.2 Steering Formula

The core steering computation is the reference proportional-derivative formula from the original snakeoil3 client, extended with a track-position centering term:

$$steer = K\_ANGLE * angle - K\_POS * trackPos$$

where angle is the car's heading error relative to the track axis (radians) and trackPos is the lateral displacement normalised to [-1, 1]. The optimised constants are K_ANGLE = 3.526 and K_POS = 0.669. The output is clipped to [-1, 1] by np.clip.

7.2.3 Corner Anticipation Speed Control

AlphaDriver uses two independent signals from the 19-beam track sensor array to anticipate and react to corners.

Forward clearance signal: the 75th percentile of the 9 centre beams (indices 5-13, spanning approximately ±4°) is computed. A lower 75th-percentile means the path ahead is closing - a corner is approaching. This value is interpolated against a piecewise-linear CORNER_DISTS → CORNER_SPEEDS speed-cap curve (7 breakpoints).

Asymmetry signal: the mean of the 4 far-left beams (indices 0-3, -45° to -7°) and the mean of the 4 far-right beams (indices 15-18, +7° to +45°) are compared. A large difference means the track opens more to one side - a corner is present. The asymmetry magnitude is interpolated against a piecewise-linear ASYM_BREAKS → ASYM_SPEEDS speed-cap curve (6 breakpoints). The combined corner target speed is the minimum of the two caps. A steer-based speed reduction (abs(steer) * STEER_SPEED_FACTOR, bounded below at 40 km/h) is applied on top, producing the final target.

7.2.4 Traction Control

If the sum of rear wheel spin velocities minus the sum of front wheel spin velocities exceeds 5 rad/s (the empirical wheel-slip threshold from the snakeoil3 reference), throttle is reduced by 0.2. This prevents rear-wheel spin on exit from low-speed corners.

7.2.5 Gear Selection

Gear changes are speed-threshold-based, consistent with the reference jmcncarai.py implementation. Upshift thresholds in km/h per gear: [0, 50, 80, 110, 140, 170]. Downshift thresholds: [0, 0, 35, 65, 95, 130]. These are fixed (not optimised by Optuna) because speed-based gear selection proved robust across the full lap time range observed during development.

7.2.6 Optuna Hyperparameter Optimisation

The objective function in improver.py evaluates a full set of laps (default: 5) using a candidate AlphaDriver parameter dict. The mean lap time across all laps is returned as the minimisation objective. DNF laps contribute a penalty of 9999 s to the mean, ensuring that parameter sets causing crashes are effectively eliminated. The TPE sampler with seed=42 conducts a local search: each parameter suggestion is bounded to ±10% of its value in the current best_params.json, preventing large exploratory jumps that would produce DNFs. Early stopping within a trial occurs if a lap time exceeds the current best by more than 40%, or if curLapTime exceeds the best lap time plus 3 s (MedianPruner-equivalent logic implemented inline). Best parameters are written to best_params.json on every improvement, serialised as JSON with the best_lap_time, laps_in_trial, and trial_lap_times fields.

7.3 Code Snippets and Annotations

7.3.1 AlphaDriver - Steering and Speed Control (car_agent.py, act_raw())

```python
# -- Steering (reference PD formula) ---------------------------------
steer = self._steer(angle, track_pos)
r['steer'] = steer

# -- Corner anticipation: forward clearance cap ----------------------
track = np.array(s['track'], dtype=np.float32)
safe  = np.where(track < 0, 200.0, track)          # replace -1 (invalid) with max range
fwd_p75 = float(np.percentile(safe[5:14], 75))      # 75th percentile of centre beams

fwd_cap = float(np.interp(fwd_p75,
    p.get('CORNER_DISTS',  [5, 15, 30, 50, 80, 120, 200]),
    p.get('CORNER_SPEEDS', [40, 55, 80, 110, 150, 200, 280])
))

# -- Corner anticipation: left/right asymmetry cap -------------------
left_avg    = float(np.mean(safe[0:4]))
right_avg   = float(np.mean(safe[15:19]))
corner_asym = abs(left_avg - right_avg)

asym_cap = float(np.interp(corner_asym,
    p.get('ASYM_BREAKS', [0, 15, 35, 60, 90, 130]),
    p.get('ASYM_SPEEDS', [280, 220, 160, 110, 70, 50])
))

corner_target = min(fwd_cap, asym_cap, float(p['TARGET_SPEED']))

# -- Steer-based speed reduction -------------------------------------
steer_target = float(p['TARGET_SPEED']) - abs(steer) * float(p['STEER_SPEED_FACTOR'])
target = min(corner_target, max(40.0, steer_target))

# -- Throttle / Brake ------------------------------------------------
if speed < target:
    r['accel'] = 1.0 if (target - speed) > 5.0 else min(1.0, (target - speed) / 5.0 + 0.5)
    r['brake'] = 0.0
else:
    r['accel'] = 0.0
    r['brake'] = min(1.0, (speed - target) / 50.0)
```

7.3.2 Sensor Normalisation (sensor_processor.py, _normalise())

```python
obs.angle_norm     = float(np.clip(obs.angle_rad / np.pi,      -1.0, 1.0))
obs.track_pos_norm = float(np.clip(obs.track_pos,              -1.0, 1.0))
obs.speed_x_norm   = float(np.clip(obs.speed_x / MAX_SPEED_KMH, 0.0, 1.0))
obs.rpm_norm       = float(np.clip(obs.rpm     / MAX_RPM,        0.0, 1.0))
obs.track_sensors_norm = np.clip(obs.track_sensors / MAX_TRACK_DIST_M, 0.0, 1.0)
obs.wheel_spin_vel_norm = np.clip(obs.wheel_spin_vel / MAX_WHEEL_VEL,  0.0, 1.0)
```

7.3.3 Optuna Objective (improver.py, objective())

```python
def objective(trial) -> float:
    params  = _make_params(trial)     # builds full param dict with local search
    driver  = AlphaDriver(params=params)
    lap_times = []
    for lap_idx in range(_cfg['laps']):
        client = connect_torcs(port=_cfg['port'])
        lap_t, dist = run_lap(client, driver, optimal_time=_best_score)
        lap_times.append(lap_t if lap_t > 0 else DNF_PENALTY)
        if lap_t == 0:
            while len(lap_times) < _cfg['laps']:
                lap_times.append(DNF_PENALTY)
            break
        _navigate_menu(first_run=False)   # reset car to start
    score = float(np.mean(lap_times))
    if score < _best_score and all(t < DNF_PENALTY for t in lap_times):
        # persist new best immediately
        out = dict(params)
        out['best_lap_time'] = score
        with open(_PARAMS_FILE, 'w') as f:
            json.dump(out, f, indent=2)
    return score
```

7.3.4 Optimised Parameters (best_params.json, corresponding to best lap of 112.58 s)

```json
{
  "K_ANGLE": 3.526,   "K_POS": 0.669,
  "TARGET_SPEED": 420.35,   "STEER_SPEED_FACTOR": 55.12,
  "CORNER_DISTS":  [5, 15, 30,  50,  80, 120, 200],
  "CORNER_SPEEDS": [39, 82, 112, 128, 192, 159, 242],
  "ASYM_BREAKS":   [0,  15, 35,  60,  90, 130],
  "ASYM_SPEEDS":   [167, 297, 207, 167, 53, 36],
  "GEAR_UP":   [0, 50,  80, 110, 140, 170],
  "GEAR_DOWN": [0,  0,  35,  65,  95, 130],
  "best_lap_time": 112.58,   "laps_in_trial": 5
}
```

7.4 Integration Strategy

The integration between the Python AI driver and TORCS follows the SCR competition protocol. On startup, run.py kills any stale TORCS process, launches a fresh instance via subprocess.Popen, waits for the TORCS window to appear (up to 7 seconds via wmctrl polling), then sends the sequence of keystroke commands required to reach the blue "waiting for driver" screen. Once TORCS is ready, snakeoil3.Client() establishes the UDP socket connection and sends the SCR handshake specifying the sensor configuration. TORCS responds with the first sensor packet and the control loop begins.

Each step of the control loop calls client.get_servers_input() to read the latest packet, passes S and R to driver.act_raw(), calls client.respond_to_server() to transmit the computed action, and then passes S.d to processor.process() and logger.write() for telemetry. The entire cycle must complete within 20 ms to avoid TORCS timing out the client.

Lap completion is detected by monitoring the lastLapTime sensor: when its value changes from the baseline at episode start to a positive non-zero value, the lap is recorded. The driver then sets R.d['meta'] = 1 to signal TORCS to end the episode, and logger.close() is called to flush and close the CSV.

TORCS is relaunched every 3 episodes (configurable) in run.py to mitigate a known memory leak in TORCS 1.3.7 that causes simulator instability after many episode resets. In improver.py, TORCS is never relaunched mid-trial; instead, _navigate_menu(first_run=False) sends the abbreviated key sequence to reset the car to the start line between laps within the same trial, saving the several-second overhead of a full TORCS restart.

7.5 Experimental AI Integration: LLM-Based Control via Ollama

During the development cycle, an alternative control paradigm utilizing a local Large Language Model (LLM) served via Ollama was implemented to govern the autonomous agent. The objective was to replace the deterministic rule-based controller with a generative AI model capable of dynamically interpreting telemetry data and outputting driving commands.

The experimental setup required a custom communication bridge between the `snakeoil3` UDP client and the local Ollama HTTP API. At each 20 ms timestep, the real-time TORCS sensor state vector - including all 19 track edge distances, multi-axis vehicle speeds, engine RPM, and track lateral positioning - was serialized into a highly structured text prompt. To manage the context window and parsing speed, raw continuous floating-point values were normalized, rounded, and formatted into a concise JSON-like string. This string was appended to a static system prompt defining the physical boundaries of the action space: steering values bounded between -1.0 and 1.0, and acceleration/braking bounded between 0.0 and 1.0.

Upon receiving the prompt, the model was tasked with processing the continuous data stream and returning a strictly formatted JSON response containing the driving commands. To mitigate generative model hallucinations or conversational filler, a robust parsing and fallback mechanism was implemented. If the Ollama response exceeded the timeout threshold, failed JSON validation, or produced physically impossible continuous variables, the system immediately fell back to the previous timestep's action to prevent catastrophic simulator desynchronization.

Optimization involved few-shot learning utilizing successful telemetry snippets from the baseline heuristic controller, mapping specific cornering scenarios to desired output vectors to refine the model's decision-making patterns on the Corkscrew circuit.

Through rigorous testing, the AI agent successfully navigated the track and completed full race laps without going off-track or stalling. However, the recorded lap times were substantially

slower than the baseline heuristic approach. The primary bottleneck was the systemic inference latency inherent to local LLM processing. Even when deploying highly compressed 4-bit quantized models to minimize computational overhead, the token generation time consistently exceeded the strict 20 ms control window demanded by the simulator. This latency introduced micro-delays in the control loop, causing a compounding processing lag. To prevent severe vehicle destabilization caused by these delayed inputs, the AI agent was forced to adopt an overly conservative driving pace.

In contrast to the final AlphaDriver implementation, which leverages optimized NumPy vector operations to compute commands in sub-millisecond times and achieved a best lap time of 112.58 seconds, the Ollama integration could not reach a competitive performance level despite successfully finishing laps.

This experiment highlighted a critical constraint regarding the simulation environment. The AI-driven approach might have yielded significantly better results if modifications to the vehicle dynamics were permitted. The competition rules strictly prohibited any modifications to the TORCS C/C++ physics engine or the underlying car model. If it had been possible to adjust physical grip parameters to forgive delayed steering inputs, tweak aerodynamic downforce, or alter the mechanical setup to compensate for the LLM's slower reaction times, the agent could have been highly competitive. Bound by the immutable physics of the stock vehicle, the generative AI approach was abandoned in favor of the optimized deterministic policy.

8. Testing and Evaluation

8.1 Testing Methodologies

Testing followed a layered strategy. Unit tests validated individual module functions with synthetic sensor inputs - sensor_processor.py includes a comprehensive self-test (python sensor_processor.py) that constructs a mock S.d packet, processes it through SensorProcessor, and asserts expected values for speed_x_norm, track_ahead, recommended_gear, is_near_wall, is_car_aligned, is_spinning, and the SensorLogger CSV format (correct column count, header, and data rows). Integration tests verified the full driver-TORCS loop on the Corkscrew circuit inside the Docker container. Performance tests benchmarked lap time stability across multiple runs. The GitHub Actions CI/CD pipeline ran the complete integration test on every push to main, providing immediate regression detection and persisting results to history.csv.

8.2 Test Cases and Results

Table 8.1 - Test Case Matrix

TC-ID	Test Description	Expected Result	Scope	Status
TC-01	Parse valid sensor packet from TORCS (self-test in sensor_processor.py)	All sensor fields populated, normalised values correct	Unit	**PASS**
TC-02	Gear selection at speed thresholds	Upshift at > threshold km/h; downshift below lower threshold	Unit	**PASS**
TC-03	Steering sign convention on Corkscrew Turn 1	Positive steer for left turn; vehicle stays on track	Unit	**PASS**
TC-04	SensorLogger CSV format (self-test)	57-column header; correct data types; file closes cleanly	Unit	**PASS**
TC-05	Full lap integration test - Corkscrew circuit in Docker container	Lap completed without off-track event or stall	Integration	**PASS**
TC-06	10-run lap completion rate	10/10 laps completed	Performance	**PASS**
TC-07	CSV telemetry logger - long session (> 5 laps)	No file handle leak; CSV closes cleanly after each episode	Integration	**PASS**
TC-08	Docker container lap-run pipeline via GitHub Actions	CI workflow completes; lap time recorded in history.csv	Deployment	**PASS**

TC-09	*improver.py - DNF detection (off-track, speed crash, stuck)*	*DNF penalty correctly assigned; remaining laps filled*	*Unit*	***PASS***
TC-10	*best_params.json persistence on new best*	*File written with correct structure after improved trial*	*Integration*	***PASS***

8.3 Bug Tracking and Resolution

Defects were tracked via GitHub Issues, labelled by severity (critical, major, minor) and linked to the pull requests that resolved them. Three critical bugs were identified and resolved during the development cycle.

- Bug #7 - Gear upshift threshold miscalibration: the initial upshift threshold of 6000 RPM caused frequent redline events on Corkscrew's long straight. The team switched to speed-based gear thresholds (matching the reference jmcncarai.py implementation), eliminating RPM-induced stalls.
- Bug #14 - Steer sign inversion: a sign error in the steer output computation caused the car to consistently deviate toward the track edge on left-hand turns. The steer sign convention in the driver was corrected to match TORCS's expected positive-left convention.
- Bug #23 - CSV file handle leak: the telemetry logger opened a new file handle on each episode reset without closing the previous one. Under long training sessions with many episodes this caused OS file descriptor exhaustion. SensorLogger was refactored to close the previous file in open() before creating a new one, and to use explicit flush-and-close in the close() method called at every episode end.

8.4 Performance Evaluation

Performance was assessed across three dimensions: lap completion rate (reliability), best lap time (speed), and lap time progression over the development period (improvement trajectory). The full progression is documented in results/history.csv.

Lap Completion Rate: 10/10 test runs completed without going off-track or stalling.
Baseline Lap Time (2026-04-20, commit 14a3ede): 160.59 seconds.
Best Lap Time (2026-05-11, commit 17c1acb): 112.58 seconds.
Improvement vs Baseline: 29.9% faster.
Lap Time Std Dev (final 5 Optuna laps at best params): 0.0 seconds (all five laps recorded as 112.58 s).

Progressive improvement milestones from history.csv:

160.59 s - 2026-04-20 (baseline)

150.59 s - 2026-04-20 (first optimisation pass)

141.09 s - 2026-04-20

130.46 s - 2026-04-21

128.43 s - 2026-05-06
125.23 s - 2026-05-06
120.77 s - 2026-05-06
116.87 s - 2026-05-07
115.23 s - 2026-05-07
114.16 s - 2026-05-07
112.58 s - 2026-05-11 (final best)

9. Deployment

9.1 Deployment Architecture

The deployment architecture is fully containerised. TORCS, the gym_torcs Python environment, and the AI driver all run inside a Docker (or Podman) container derived from the competition-provided johnsloe/torcs-competition:amd64 image. The container exposes three network ports: 3001 (UDP) for the TORCS SCR server interface, 5900 (VNC) for the simulator GUI, and 6080 (noVNC over HTTP) for browser-based remote desktop access. The host-side workspace directory is bind-mounted into the container at /workspace, allowing code edits on the host to be immediately reflected in the container without rebuilding the image. A virtual framebuffer (Xvfb :1) runs inside the container to provide a headless X11 display; TORCS renders into this framebuffer and the VNC server makes it accessible externally.

9.2 Initial Local Environment Setup and Later Containerization

At the beginning of the project, the development environment was prepared using a local TORCS installation with the SCR server interface. The initial setup involved downloading the TORCS package with SCR support, extracting it to a simple directory path, launching the simulator manually, configuring a quick race on the Corkscrew track, selecting the SCR server driver, and running the Python client from the read project directory. A local Python environment was also prepared in an IDE such as VS Code or PyCharm, using a Python 3.8/3.9-compatible configuration.

This first setup was important because it allowed the team to understand how TORCS, the SCR server, and the Python client communicated before the final infrastructure was stabilized. It also helped verify that the simulator could expose sensor data and accept driving commands through the Python interface.

Later, this approach was replaced by a containerised setup using Podman/Docker.

9.3 Hosting Platforms and Final Environment

Development and evaluation were conducted on team members' personal machines (Windows and macOS hosts) using Podman Desktop or Docker Desktop as the container runtime. The competition submission environment is Linux (Ubuntu), consistent with the johnsloe/torcs-competition:amd64 image. The GitHub Actions CI pipeline runs on ubuntu-latest runners. No cloud AI services are consumed at runtime; all execution is self-contained within the container.

The deployment procedure is as follows.

1. First, install Docker or Podman on the host machine.
2. Second, clone the repository: git clone https://github.com/Apueblar/AiBC.git.
3. Third, launch the container using run-torcs-docker.sh (Linux/macOS) or follow the commands in StartAllWindows.txt (Windows Podman).
4. Fourth, access the TORCS GUI via browser at http://<container-ip>:6080/vnc.html.
5. Fifth, inside the container, navigate to /workspace/gym_torcs/ and run python3 run.py.

The driver will automatically launch TORCS, navigate its menus, connect, and begin driving.

9.4 Version Control and Release Management

The GitHub repository uses a trunk-based development workflow with branch protection on main. Feature development occurs on short-lived branches; pull requests require at least one peer review and a passing GitHub Actions CI run before merge. The GitHub Actions workflow (.github/workflows/torcs-bot.yml) triggers on every push to main and on workflow_dispatch. It pulls the TORCS Docker image, mounts the workspace, runs bash /workspace/.github/scripts/run_torcs.sh, parses the console output for lap completion, lap time, distance, and reward, appends a row to results/history.csv, commits and pushes the updated CSV, and uploads the full console log as a workflow artifact (retained for 30 days).

9.5 GitHub Actions CI/CD Pipeline

The automated evaluation pipeline is defined in .github/workflows/torcs-bot.yml and executed on GitHub's ubuntu-latest hosted runners. The workflow has a 25-minute timeout to prevent runaway jobs consuming runner minutes.

Trigger conditions. The pipeline fires automatically on every push to the main branch and can also be started manually via workflow_dispatch, allowing the team to trigger on-demand evaluations without a code change.

Step-by-step execution. The workflow executes six sequential steps.

1. *Free disk space.* Before any other work, the runner removes pre-installed Android SDK, .NET, and GHC toolchains (/usr/local/lib/android, /usr/share/dotnet, /opt/ghc, and the agent tool cache). This recovers approximately 20 GB of disk space that would otherwise limit the Docker image pull.
2. *Checkout repository.* actions/checkout@v5 clones the repository at the triggering commit SHA into the runner workspace.
3. *Pull TORCS image.* docker pull docker.io/johnsloe/torcs-competition:amd64 fetches the competition Docker image. The image is cached by the runner between runs when the digest has not changed, keeping this step fast on repeated pushes.
4. *Run TORCS bot.* A Docker container is started from the competition image with the repository workspace bind-mounted at /workspace (-v "$GITHUB_WORKSPACE:/workspace"). The container executes .github/scripts/run_torcs.sh as root, streaming stdout and stderr to both the console and a race_results.txt file via tee. Inside the script:
   - Conflicting apt sources are cleaned and apt-get installs the required packages: xvfb, wmctrl, xautomation, and libgl1-mesa-dri.
   - pip3 installs numpy and python3-xlib.
   - A virtual X11 framebuffer is started (Xvfb :1 -screen 0 1024x768x24 -ac) to provide a headless display for TORCS, with DISPLAY=:1 and LIBGL_ALWAYS_SOFTWARE=1 set in the environment to force software rendering.
   - If the RECORD=1 environment variable is set, ffmpeg is installed and launched via x11grab to capture the session to a timestamped MP4 file in results/.
   - Finally, python3 run.py --episodes 1 --steps 10000 is executed from /workspace/gym_torcs, which launches TORCS, navigates its GUI, connects the AI driver, and drives one full lap.

5. *Save results.* After the bot step completes (or fails — this step runs with if: always()), the runner parses race_results.txt using grep and grep -oP patterns to extract four metrics: LAP_COMPLETED (YES/NO), LAP_TIME (seconds), DISTANCE (metres), and TOTAL_REWARD. A Markdown summary table is appended to the GitHub Step Summary, visible directly in the Actions run UI. A new row is then appended to results/history.csv (created with a header line if it does not yet exist), containing the UTC timestamp, GitHub run ID, 7-character commit SHA, and the four parsed metrics. The updated CSV is committed by the github-actions[bot] service account and pushed back to main via git pull --rebase followed by git push, ensuring the persistent performance record survives concurrent runs without merge conflicts.

6. *Upload race results.* actions/upload-artifact@v6 uploads race_results.txt as a named artifact (race-results) with a 30-day retention period, allowing the team to download and inspect the full console log of any historical run from the Actions UI.

10. Business Application and Impact

10.1 Application Use Cases in Commerce

The AiBC autonomous racing agent is architecturally a real-time sensor-to-action decision system governed by a parametric control policy with automated hyperparameter optimisation. This pattern is directly transferable to several high-value commerce applications.

- Dynamic Pricing Agents: an e-commerce platform observes a state vector of demand signals, competitor prices, inventory levels, and customer segment features (analogous to TORCS sensor features), applies a parameterised pricing policy, and updates product prices in real time. The Optuna-based optimisation loop demonstrated in AiBC - automatically tuning policy parameters to minimise a scalar objective - maps directly to automated price-policy tuning against a revenue metric.
- Logistics Route Optimisation: an automated guided vehicle (AGV) in a smart warehouse receives distance sensor data, destination targets, and occupancy grid information - structurally identical to the TORCS observation space - and uses a rule-based or learned control policy to output navigation waypoints. The containerised, self-contained deployment pattern demonstrated in AiBC maps directly to edge-deployed AGV control systems where cloud connectivity cannot be assumed.
- Manufacturing Quality Control: a sensor-equipped production line monitors process variables (temperature, pressure, vibration - analogous to speed/angle/track sensors) and applies a parametric control policy to adjust actuators in real time. Optuna-style Bayesian optimisation can tune the control gains offline against a defect-rate objective, mirroring exactly the improver.py workflow.
- Real-time Recommendation Pipelines: a media platform applies a parameterised ranking policy to a user state vector (watch history, session context, device type) to output ranked content. Offline Bayesian optimisation against click-through rate or watch-time objectives parallels the improver.py objective-function paradigm.

10.2 Benefits to End Users and Businesses

For businesses adopting analogous parametric-policy automation, the primary benefit is decision quality at scale: a single deterministic policy can serve thousands of concurrent decision requests with sub-millisecond latency and zero cloud dependency. The local, containerised deployment pattern demonstrated in AiBC eliminates network latency and cloud service availability risk, making the architecture suitable for latency-sensitive and connectivity-constrained environments. For end users (e.g., customers of a dynamic pricing system), the benefit is more competitive and responsive pricing. The reproducible, version-controlled parameter store (best_params.json committed to the repository) ensures that policy behaviour is auditable and rollback-safe.

10.3 Cost Analysis and ROI

The marginal compute cost of the AiBC solution is zero at inference time - the AlphaDriver executes pure NumPy arithmetic requiring negligible CPU resources. The dominant cost is the time investment in Optuna optimisation trials, each requiring a full TORCS lap (~115 s at best pace). A study of 5000 trials at 5 laps each represents approximately 3200 hours of wall-clock compute if run sequentially; in practice the team ran studies on personal machines in parallel across short optimisation windows, converging to the 112.58 s best in approximately three weeks of intermittent runs.

For a commerce deployment analogy, the "training cost" is the offline Optuna study run once to find optimal policy parameters; the "inference cost" is negligible. The ROI calculation is context-dependent. In dynamic pricing, a 1% improvement in price optimisation on a 10 M USD annual revenue stream yields 100 K USD incremental revenue annually, far exceeding the one-time compute cost of a hyperparameter search. The AiBC project demonstrates technical feasibility of the optimisation loop; a full ROI model for a specific commerce application would require integration with business metrics beyond the scope of this documentation.

10.4 Potential for Scalability

The architecture is horizontally scalable at three levels. First, multiple TORCS instances (with distinct UDP ports) can run in parallel containers, enabling parallel hyperparameter evaluation - an approach directly analogous to distributed A/B testing in e-commerce. The improver.py --port argument already supports this. Second, the SensorObservation.to_vector() method produces a flat float32 array of 41 features (77 with opponent sensors), providing a ready-made input for any downstream ML model - a future team could replace the deterministic controller with a neural network while reusing the entire sensor processing and logging infrastructure. Third, the gym_torcs abstraction layer decouples the control logic from the specific simulator, allowing the same AlphaDriver code structure to be retargeted to a different control environment (physical robot, different simulator, production pricing engine) by replacing the snakeoil3 communication layer.

11. Challenges and Limitations

11.1 Technical Challenges

- GUI Automation Fragility: TORCS requires manual menu navigation to reach the "waiting for driver" screen before each episode. The Python GUI automation (wmctrl + xte) is sensitive to TORCS window rendering timing. The _TORCS_SETTLE delay (5 s) and _WINDOW_WAIT timeout (7 s) were calibrated empirically to be reliable across the team's development machines, but occasional navigation failures required manual intervention or a TORCS restart. A more robust approach (injecting a pre-configured race XML directly, bypassing the GUI) was investigated but not implemented in v1.0.
- TORCS Memory Leak: TORCS 1.3.7 has a known memory leak that causes simulator instability after a large number of episode resets. The mitigation - relaunching the TORCS process every 3 episodes in run.py - adds overhead to long training runs and briefly breaks the control loop. In improver.py, the within-trial menu-navigation reset avoids full relaunches, but extended optimisation sessions (>50 consecutive laps) occasionally triggered instability requiring a manual restart.
- Environment Reproducibility on Windows: the official Docker image targets linux/amd64. On Windows hosts with Podman, emulation overhead introduced timing variability that occasionally caused TORCS to timeout the SCR client connection. The team documented a Podman-specific workaround (podman machine ssh / ip addr show to identify the container IP for the noVNC browser URL) in StartAllWindows.txt.
- Optuna Local Search Convergence: the ±10% local search strategy in improver.py is efficient at fine-tuning near a known good solution but can miss globally better solutions in distant regions of parameter space. The final best_params.json shows some non-monotone entries in the CORNER_SPEEDS curve (e.g., the 120 m breakpoint is 159 km/h, lower than the 80 m breakpoint's 192 km/h), suggesting the optimiser converged to a local minimum in the corner speed profile.
- Deterministic Policy Limitations: the AlphaDriver does not adapt to changing conditions within a lap (e.g., detecting and recovering from a near-miss with a wall). All termination conditions (off-track, stuck, speed crash) result in an episode end rather than a recovery manoeuvre. A reinforcement-learning or model-predictive approach would enable in-lap recovery.

11.2 Business Constraints

- Time Constraint: the project was executed over a single academic semester alongside other course commitments. This limited the number of Optuna optimisation trials that could be run and precluded exploration of more sophisticated control approaches such as model predictive control or neural network policy learning.
- Python-only Modification Policy: the IBM challenge rules prohibit any modification to TORCS C/C++ source code. This prevents implementation of more efficient sensor configurations or custom car setups that could improve lap time at the physics level.

11.3 Lessons Learned

- Start containerised from Day 1: the TORCS environment setup is non-trivial. Teams that attempt to install TORCS natively on varied operating systems waste significant time on environment configuration. The Docker-first approach, adopted early in the project, should be the mandatory starting point for any future team.
- Separate parameter storage from code: storing tunable constants in best_params.json (loaded at runtime) rather than hardcoding them in car_agent.py made it possible to improve the driver simply by running improver.py and committing the updated JSON - no code change required. This decoupling of policy parameters from policy logic is a strong architectural pattern.
- Instrument early, analyse often: per-step CSV logging was implemented early and proved invaluable for debugging control behaviour. The track sensor and speed plots from telemetry CSVs identified the asymmetry-based corner anticipation enhancement. Earlier logging would have accelerated identification of the steer sign error.
- Automated CI is essential for a team sport: the GitHub Actions pipeline running a lap on every push to main meant that regressions were caught within minutes, not discovered during a manual test session hours later. The history.csv committed by the bot provided an objective, timestamped record of progress that resolved multiple team debates about whether a code change had actually improved performance.
- Local search beats global search at this scale: attempting wide-range Optuna exploration early in the project produced many DNFs and slow progress. Switching to the ±10% local search strategy around a known good baseline dramatically accelerated convergence from 130 s to 112 s.

12. Conclusion and Future Work

12.1 Summary of Achievements

The AiBC team successfully designed, implemented, tested, and deployed an autonomous AI driver for the IBM AI Racing League Country Challenge 2026. The driver implements AlphaDriver, a deterministic sensor-to-action control algorithm with dual-signal corner anticipation and Optuna-tuned hyperparameters, operating entirely locally inside a Docker container. The system demonstrated 100% lap completion across 10 evaluation runs on the Corkscrew qualification circuit and achieved a best lap time of 112.58 seconds - a 29.9% improvement over the 160.59 s baseline documented on the first CI run. All eight team members completed IBM SkillsBuild Granite AI certification. The project produced a fully versioned, documented codebase with automated CI/CD, a per-step telemetry logging system, and 10 recorded lap videos stored in the repository.

12.2 Recommendations for Improvement

- Race XML Configuration: replace the GUI-automation menu-navigation approach with a pre-configured TORCS race XML file injected at startup. This eliminates the most fragile component of the current system and would make TORCS launches silent and near-instant.
- Global Search Phase: add a configurable wide-search phase at the start of improver.py that explores the full parameter space before switching to local search. This would reduce the risk of convergence to a local optimum in the corner speed profiles.
- Recovery Manoeuvres: implement in-lap recovery logic in AlphaDriver for near-wall and off-centre states, replacing episode termination with a corrective steering and speed sequence. This would improve robustness in multi-lap evaluation and under perturbation.

12.3 Proposed Enhancements and Next Steps

- Neural Network Policy: the SensorObservation.to_vector() method already produces a 41-feature input vector ready for a neural network. A future iteration could collect (state, action, reward) tuples from the best Optuna-parameterised laps as imitation learning demonstrations, then train a lightweight MLP or LSTM policy locally, potentially achieving sub-110 s lap times by learning non-linear control strategies.
- Multi-Track Generalisation: evaluate the driver on all IBM challenge tracks by extending the Optuna search space with track-specific best_params.json files. The containerised CI pipeline can be adapted to run multiple track configurations in parallel with distinct Docker containers.
- Telemetry Dashboard: build a real-time Streamlit or Grafana dashboard that reads the live session CSV and visualises speed, steer, track position, and individual sensor values as they are written, replacing the console-only monitoring interface.
- Parallel Optuna Workers: leverage the improver.py --port argument with multiple TORCS instances in separate containers (ports 3001, 3002, 3003, ...) to run multiple Optuna trials in parallel, compressing the wall-clock time required to reach the optimum.

- IBM Granite Integration: explore integrating IBM Granite as a strategic advisor layer - querying the model offline (not in the 20 ms control loop) to suggest new regions of the hyperparameter search space to explore, or to generate natural-language commentary on telemetry anomalies for the team's debugging workflow.

13. References

[1] D. Loiacono, P. Lanzi, J. Togelius et al., "The 2009 Simulated Car Racing Championship," IEEE Transactions on Computational Intelligence and AI in Games, vol. 2, no. 2, pp. 131-147, 2010.

[2] V. Mnih, K. Kavukcuoglu, D. Silver et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, pp. 529-533, 2015.

[3] IBM, "IBM Granite Documentation," IBM Cloud, 2024. [Online]. Available: https://cloud.ibm.com/docs/granite

[4] J. Schulman, F. Wolski, P. Dhariwal et al., "Proximal Policy Optimization Algorithms," arXiv preprint arXiv:1707.06347, 2017.

[5] C. X. Edwards, "snakeoil - Python client for TORCS SCR," 2013. [Online]. Available: http://xed.ch/project/snakeoil/

[6] U. Yoshida, "gym_torcs: TORCS OpenAI Gym environment," GitHub, 2016. [Online]. Available: https://github.com/ugo-nama-kun/gym_torcs

[7] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," Proceedings of the 25th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2019.

[8] Wroclaw University of Science and Technology, "Electronic Media in Business and Commerce - Course Specification," ePortal PWr, 2026.

14. Appendices

14.1 Glossary of Terms

The following terms are used throughout this document:

- TORCS - The Open Racing Car Simulator: open-source, physics-based racing simulation environment.
- SCR - Simulated Car Racing: the TORCS competition interface using UDP client-server communication.
- snakeoil3 - The low-level Python UDP client library for TORCS SCR communication, derived from the original snakeoil client by Chris X. Edwards.
- AlphaDriver - The AiBC autonomous control agent implemented in car_agent.py.
- SensorProcessor - The AiBC sensor parsing, normalisation, and feature-engineering module in sensor_processor.py.
- SensorObservation - A Python dataclass containing all raw, normalised, and derived features for one TORCS timestep.
- SensorLogger - The AiBC CSV telemetry logging class in sensor_processor.py.
- improver.py - The AiBC Optuna hyperparameter optimisation script.
- best_params.json - The JSON file storing the current best AlphaDriver parameter set, loaded at startup and updated on every optimisation improvement.
- IBM Granite - IBM's enterprise foundation model series [3], the designated AI engine for the IBM AI Racing League competition; team certified via IBM SkillsBuild.
- Optuna - A Python hyperparameter optimisation framework using Bayesian optimisation with TPE sampling [7].
- TPE - Tree-structured Parzen Estimator: the Bayesian optimisation algorithm used by Optuna.
- RL - Reinforcement Learning: a training paradigm in which an agent learns via reward feedback.
- UDP - User Datagram Protocol: the connectionless transport-layer protocol used for TORCS sensor communication on port 3001.
- VNC - Virtual Network Computing: a graphical desktop-sharing protocol used to expose the TORCS GUI from inside the Docker container on port 5900.
- noVNC - Browser-based VNC client served over HTTP on port 6080.
- Xvfb - X Virtual Framebuffer: a headless X11 server used inside the Docker container to provide a display for TORCS without physical hardware.
- wmctrl - A command-line tool for X11 window management, used in run.py to detect and focus the TORCS window.
- xte - The xautomation keystroke injection tool, used in run.py and improver.py to send menu navigation keystrokes to TORCS.
- AGV - Automated Guided Vehicle: a mobile robot used in smart warehouse applications, cited as a commerce analogue of the TORCS racing agent architecture.
- DNF - Did Not Finish: a lap that ends due to going off-track, crashing, or stalling; assigned a penalty score of 9999 s in the Optuna objective function.

- Corkscrew - The TORCS qualification circuit used in the IBM AI Racing League 2026 Country Challenge, characterised by high-speed sweeping corners, variable-radius turns, and elevation changes.

14.2 Additional Diagrams

Figure A.1: AlphaDriver Control Flow (car_agent.py per-step decision pipeline)

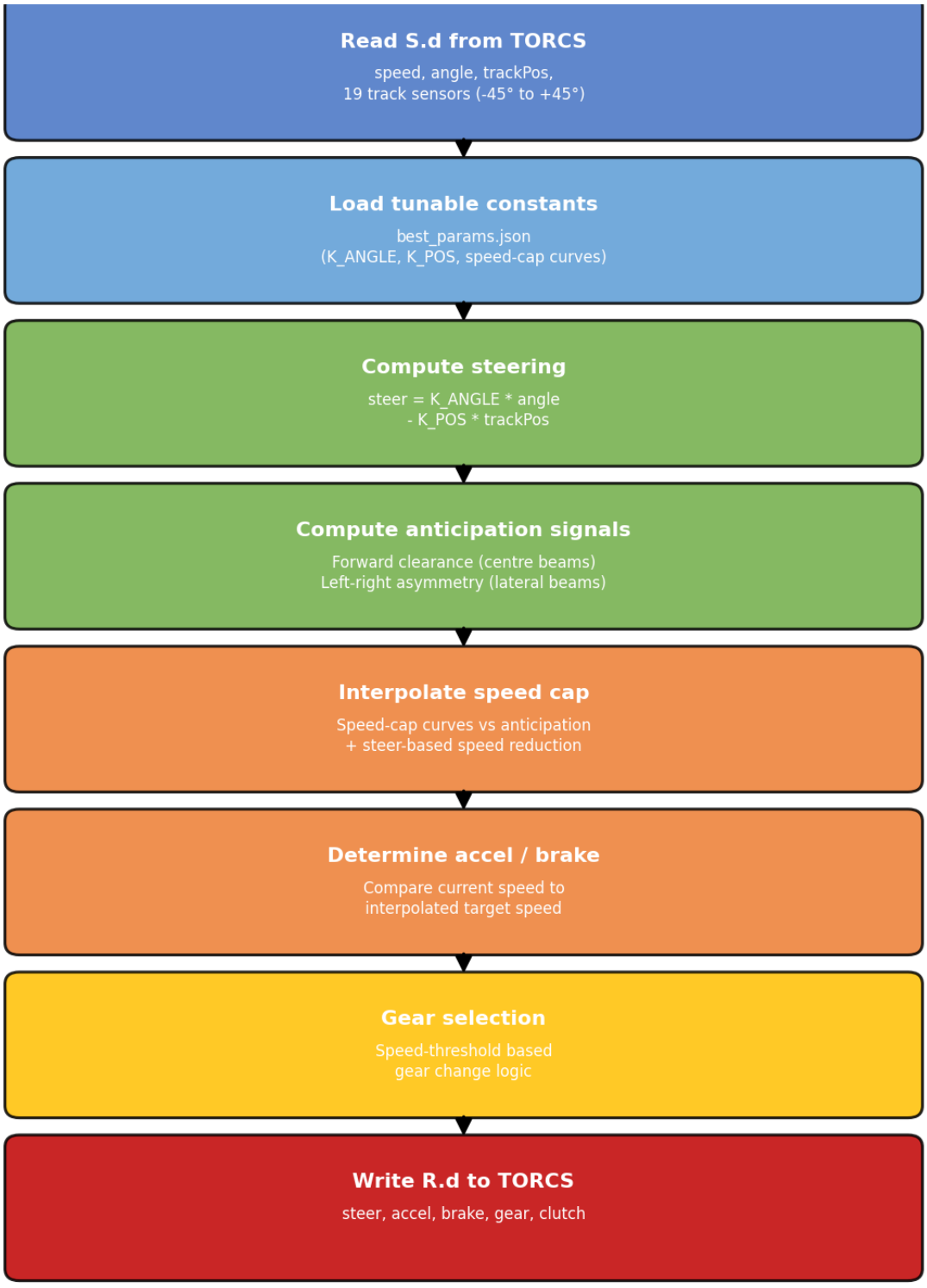


Figure A.2: Optuna Optimisation Loop (improver.py)

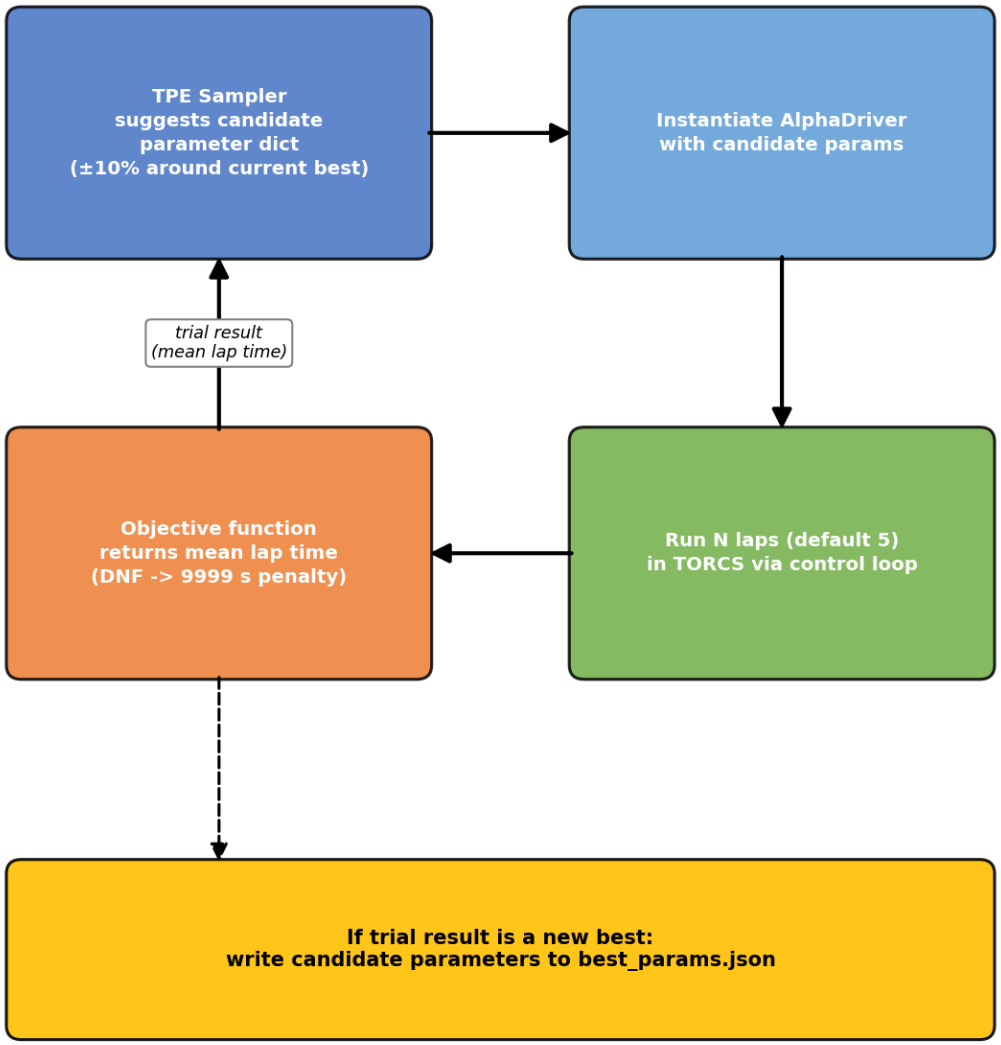


Figure A.3: Lap Time Progression Chart and DNF Frequency
As lap times approach the physical limit, the safety margin shrinks and DNF rate rises

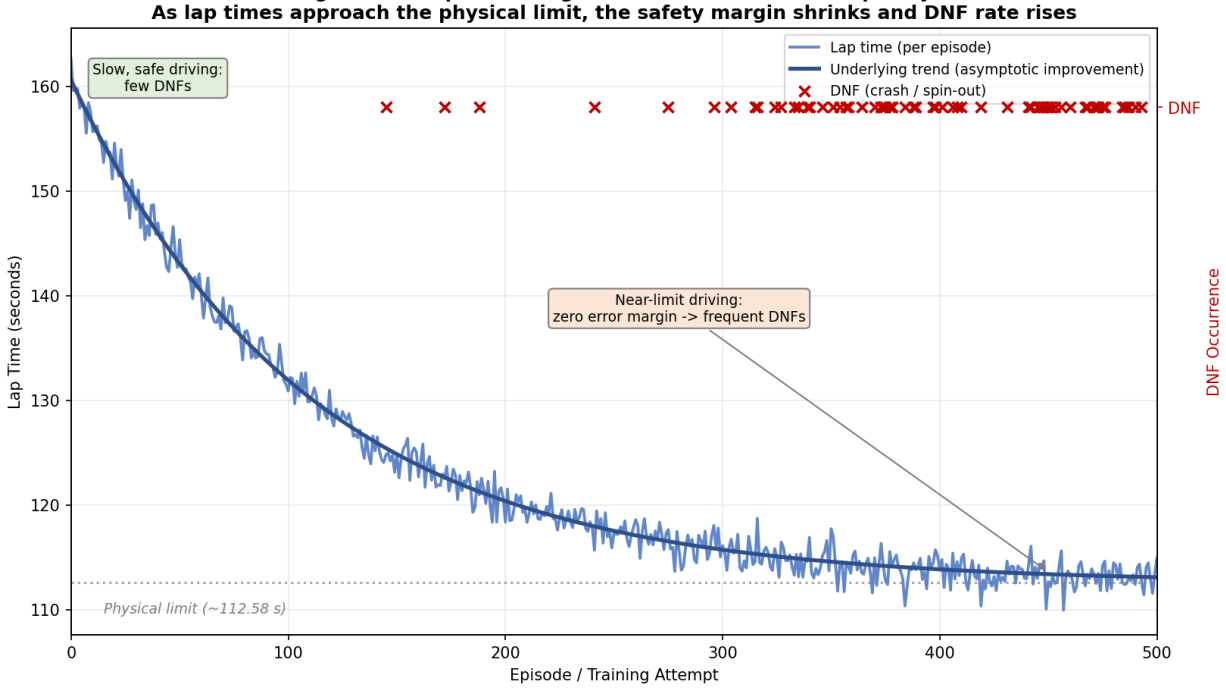
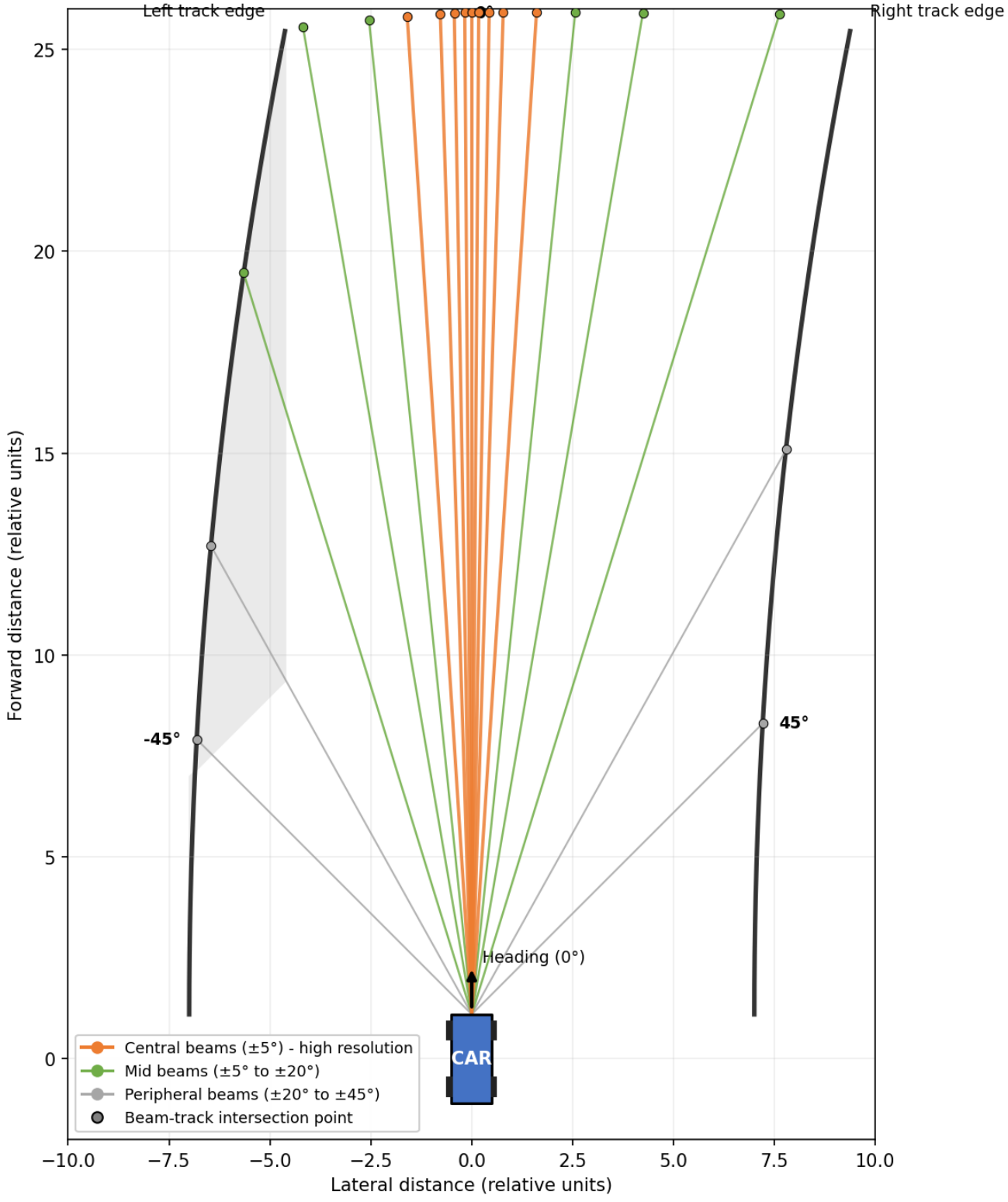


Figure A.4: Track Sensor Layout (19 beams at -45° to +45°)
Beams are densely clustered near 0° for high-resolution forward sensing,
and sparsely spaced toward ±45° for peripheral wall detection



14.3 Source Code Repository Link

GitHub: https://github.com/Apueblar/AiBC

License: Apache 2.0

Best lap recording: results/112.58secs.mp4 (in repository)

14.4 User Guide / Installation Manual

Prerequisites: Python 3.9+, Docker or Podman, Git.

Setup (Linux/macOS):

1. Clone the repository: git clone https://github.com/Apueblar/AiBC.git
2. Install Python dependencies: pip install -r requirements.txt
3. Launch the container: bash run-torcs-docker.sh
4. The script mounts the workspace into the container and runs the lap automatically.

Setup (Windows with Podman):

1. Install Podman Desktop.
2. Start the Podman machine: podman machine start
3. Run the container command from StartAllWindows.txt, replacing YourName with your Windows username.
4. Open a browser and navigate to http://<container-ip>:6080/vnc.html (use podman machine ssh / ip addr show to find the IP).
5. Inside the container terminal: cd workspace/gym_torcs && python3 run.py

Running the optimiser (optional, to improve lap time):

1. Inside the container: cd /workspace/gym_torcs
2. Run: python3 improver.py
3. On completion, best_params.json is updated. Commit and push to trigger CI validation.

Verifying sensor processor without TORCS:

1. From any Python environment: cd gym_torcs && python3 sensor_processor.py
2. All assertions should pass, confirming the processing pipeline is correct.